

Lab Exercise - Cross-Architectural Analysis

Type/Family: AcidRain Wiper (MIPS32)

SHA-256: 9B4DFACA873961174BA935FDDAF696145AFE7BBF5734509F95FEB54F3584FD9A

Open the provided .bndb database with BinaryNinja and make yourself familiar. Alternatively, use BinaryNinja Cloud and instead of uploading the malware sample upload the .bndb for analysis and work from there. This is a stripped MIPS 32-bit executable, and the BNDDB provides some context, specifically since I've assigned names to a swath of library functions. We see a clear divide between application code and library code, as the libc object starts prominently right after the main function.

AcidRain is the wiper malware behind the ViaSat hack at the beginning of Russia's war against Ukraine. Its technically a modem wiper, that is designed to target ViaSat KA-SAT modems used for satellite communication. The attack went more widespread than apparently intended, and also affected close to 6000 German Enercon wind turbines, as their remote communications ability was rendered inoperable.

Lab Exercise

Challenge 1: Lets get familiar with MIPS32 and see how well our understanding transfers. Skills needed are Linux execution environment basics like what is a syscall and what is libc, MIPS basics like what registers have what purpose, how is the SYS-V calling convention implemented on MIPS, understanding of basic branch, load/store and arithmetic instructions. Also, keep in mind the delay slot execution mechanism which comes into play with jump and branch instructions.

Reference the training slides to refresh your memory. With all that in mind, have a look at this executable's start function. Can you see what arguments are handed over to crt_main_runtime? What registers are at play and what is their purpose?

Challenge 2: Navigate to sys_reboot and explain how the syscall calling convention works here. It helps to understand what arguments reboot accepts.

Challenge 3: At the beginning of the main function we see a typical Linux daemonization procedure (code **prior to** loc_4016C4). Try to read the assembly and describe the procedure to explain how AcidRain maneuvers its own process to run in the background. It helps to focus on the system calls used in this procedure, Google is your best friend if the order of syscalls otherwise doesn't ring a bell.

Bonus challenge: Reconstruct the data wiping code flow of AcidRain (code following loc_4016C4). Function sub_400688 is particularly interesting.